```
"""
Daily SMS Scheduler - Hypertension Adherence Tool
Sends personalized daily check-in links to enrolled patients via Twilio
"""

import os
import hmac
import hashlib
import time
from datetime import datetime, date
from typing import Optional

# pip install twilio schedule flask --break-system-packages
from twilio.rest import Client
import schedule
from flask import Flask, request, jsonify, abort

# ── CONFIG ──────────────────────────────────────────────────────────────
TWILIO_ACCOUNT_SID = os.environ.get("TWILIO_ACCOUNT_SID")
TWILIO_AUTH_TOKEN  = os.environ.get("TWILIO_AUTH_TOKEN")
TWILIO_FROM_NUMBER = os.environ.get("TWILIO_FROM_NUMBER")   # e.g. +15551234567
APP_BASE_URL       = os.environ.get("APP_BASE_URL")         # e.g. https://yourap
LINK_SECRET        = os.environ.get("LINK_SECRET", "change-this-secret-key")
SMS_SEND_HOUR      = int(os.environ.get("SMS_SEND_HOUR", 9))  # 9 AM daily

app = Flask(__name__)
twilio_client = Client(TWILIO_ACCOUNT_SID, TWILIO_AUTH_TOKEN)

# ── IN-MEMORY PATIENT STORE (replace with your DB / Base44 API call) ────
# Each patient: { phone, patient_id, clinic_id, name, active }
PATIENTS = [
    # Example — replace with real data source
    {
        "phone":      "+15559876543",
        "patient_id": "PT-001",
        "clinic_id":  "CLINIC-A",
        "name":       "Maria",
        "active":     True,
    },
]

# ── SECURE LINK GENERATION ──────────────────────────────────────────────
def generate_token(patient_id: str, clinic_id: str, date_str: str) -> str:
    """HMAC token — unique per patient per day, expires after 24h."""
```

```
    payload = f"{patient_id}:{clinic_id}:{date_str}"
    token = hmac.new(LINK_SECRET.encode(), payload.encode(), hashlib.sha256).hexd
    return token[:32]  # 32-char hex token

def build_checkin_url(patient_id: str, clinic_id: str) -> str:
    today = date.today().isoformat()
    token = generate_token(patient_id, clinic_id, today)
    return f"{APP_BASE_URL}/checkin?pid={patient_id}&cid={clinic_id}&date={today}

def verify_token(patient_id: str, clinic_id: str, date_str: str, token: str) -> b
    expected = generate_token(patient_id, clinic_id, date_str)
    return hmac.compare_digest(expected, token)

# ── SMS SENDING ──────────────────────────────────────────────────────────
def send_checkin_sms(patient: dict) -> bool:
    if not patient.get("active"):
        return False

    link = build_checkin_url(patient["patient_id"], patient["clinic_id"])
    name = patient.get("name", "there")

    message_body = (
        f"Hi {name} 👋 It's your daily check-in from your care team.\n\n"
        f"Take 2 minutes to let us know how you're doing:\n{link}\n\n"
        f"Reply STOP to opt out."
    )

    try:
        msg = twilio_client.messages.create(
            body=message_body,
            from_=TWILIO_FROM_NUMBER,
            to=patient["phone"],
        )
        print(f"[{datetime.now()}] SMS sent to {patient['patient_id']} | SID: {ms
        return True
    except Exception as e:
        print(f"[{datetime.now()}] ERROR sending to {patient['patient_id']}: {e}"
        return False

def send_all_daily_sms():
    print(f"\n[{datetime.now()}] Starting daily SMS batch...")
    sent = sum(send_checkin_sms(p) for p in PATIENTS if p["active"])
    print(f"[{datetime.now()}] Batch complete. {sent}/{len(PATIENTS)} sent.")

# ── API ENDPOINTS ────────────────────────────────────────────────────────
@app.route("/api/enroll", methods=["POST"])
def enroll_patient():
```

```
    """Enroll a new patient (called from Base44 when staff registers a patient)."
    data = request.json
    required = ["phone", "patient_id", "clinic_id", "name"]
    if not all(k in data for k in required):
        return jsonify({"error": "Missing required fields"}), 400

    patient = {
        "phone":      data["phone"],
        "patient_id": data["patient_id"],
        "clinic_id":  data["clinic_id"],
        "name":       data["name"],
        "active":     True,
    }
    PATIENTS.append(patient)

    # Send a welcome SMS immediately
    link = build_checkin_url(patient["patient_id"], patient["clinic_id"])
    twilio_client.messages.create(
        body=(
            f"Hi {patient['name']}! Welcome to your post-discharge care program.
            f"You'll hear from us daily. Here's today's check-in:\n{link}"
        ),
        from_=TWILIO_FROM_NUMBER,
        to=patient["phone"],
    )
    return jsonify({"status": "enrolled", "patient_id": data["patient_id"]}), 201


@app.route("/api/verify-link", methods=["GET"])
def verify_link():
    """Called by Base44 check-in page to verify the token is valid."""
    pid   = request.args.get("pid")
    cid   = request.args.get("cid")
    dt    = request.args.get("date")
    token = request.args.get("token")

    if not all([pid, cid, dt, token]):
        return jsonify({"valid": False, "reason": "Missing parameters"}), 400

    if not verify_token(pid, cid, dt, token):
        return jsonify({"valid": False, "reason": "Invalid or expired link"}), 40

    return jsonify({"valid": True, "patient_id": pid, "clinic_id": cid, "date": d


@app.route("/api/send-now", methods=["POST"])
def trigger_manual_send():
```

```
    """Manually trigger SMS batch (admin use only)."""
    secret = request.headers.get("X-Admin-Secret")
    if secret != os.environ.get("ADMIN_SECRET"):
        abort(403)
    send_all_daily_sms()
    return jsonify({"status": "sent"})


# ─── SCHEDULER ──────────────────────────────────────────────────────────
def run_scheduler():
    import threading
    schedule.every().day.at(f"{SMS_SEND_HOUR:02d}:00").do(send_all_daily_sms)
    def loop():
        while True:
            schedule.run_pending()
            time.sleep(30)
    thread = threading.Thread(target=loop, daemon=True)
    thread.start()
    print(f"Scheduler running — SMS will send daily at {SMS_SEND_HOUR}:00")


# ─── ENTRY POINT ────────────────────────────────────────────────────────
if __name__ == "__main__":
    run_scheduler()
    app.run(host="0.0.0.0", port=8000, debug=False)
```